

n in Snowflake. |
 2022-10-01 - 2025-01-01 |
 | 4. | s3: Pseudonymize pageurlpath in Snowflake and s3 bucket | Snowplow | Exposed data for the pageurlpath column in the s3 bucket.
 | 2022-10-01 - 2025-01-01 |
 | 5. | AI Gateway SSOT data source missing namespaces in events related to 'requestduochatresponse' | Snowplow | Wrong namespace assigned for requestduochatresponse | 2024-08-28 - 2024-12-04 |
 | 6. | Submitting Service Ping causes validation error in Version app for v17.6 SM instances | Snowplow | Lower than usual amount of successful Service Ping and large number of submission fails | 2024-10-04 - 2024-12-16 |
 |

 ## SECTION: enterprise-data/data-governance/data-catalog/_index.md

Purpose

The purpose of this Data Catalog is to help you understand and make use of the data that is available in our Snowflake Data Warehouse for reporting and analysis. The Data Catalog is available in our Internal GitLab Handbook.

 ## SECTION: enterprise-data/how-we-work/_index.md

*<i class="fas fa-map-marked-alt fa-fw -text-purple"></i>*Quick Links

- Calendar
- MR Roles and Responsibilities
- New Data Source
- Planning Drumbeat
- Triage
- Data Team Onboarding
- Data Team Programs

Practical guide to contributing to the Data Team Projects

Looking to get hands-on with data at GitLab? Check out our Practical Guide designed for all team members.

This guide complements our existing resources, but with a more practical focus on the step-by-step process of contributing to the Data Team's projects.

How We Work

We're happy to help you achieve your goals with Data. As a central shared service with finite time and capacity and with a responsibility to operate and develop the company's central Enterprise Data, the Data Team must focus its time and energy on initiatives that will yield the greatest positive impact to the overall global organization towards improving customer results.

Work Categorization and Prioritization

The Data Team strives to spend the majority of its time developing and operating the Enterprise Data Platform and related systems, keeping fresh data flowing through the system, regularly expanding the breadth of data available for analysis, and delivering high-impact strategic projects. We categorize our work using the framework outlined below.

Work Category	Description	Prioritization Method
Production Maintenance	Activities required to maintain efficient and reliable data services, including triage, bug fixes, and patching to meet established Service Level Objectives.	Based on severity and impact
Data Team OKRs	The Data Team identifies strategic-level OKRs in collaboration with partner teams each quarter. Prioritized through the Data Steering Committee and committed to during our quarterly planning process	
Business Operations	Data work and business engagement that is requested on an ad-hoc basis throughout the quarter. Includes foundational work to mature the Enterprise Data Ecosystem.	Prioritized on an ongoing basis and committed to during our iteration planning process.

The allocation of capacity across Production Maintenance, Data Team OKRs, and Business Operations will be determined by each pillar on a quarter-by-quarter basis in the planning file. The target allocation varies by data team pillar and needs of the business. The allocation will consider the amount of Production Maintenance support needed from the pillar, strategic initiatives that the pillar needs to support, and business operations projects that require support. Within each Work Category, issues are prioritized independently and can use the scoped Priority Label with priorities 1, 2, or 3 as options.

We use scoped labels in GitLab to track and prioritize our issues across these work categories.

In addition to the above work categories focused on operating and developing the Enterprise Data Platform and related systems, the Enterprise Data Team spends time each quarter on learning and experimentation. This time is used to learn new skills, experiment with new technologies, and improve the data program. These learning and experimentation issues are prioritized between the team member and their manager while considering Individual Growth Plans and ways to improve the data program.

Project Intake

Here's the process to follow to create a new Data issue:

1. Open a New Issue in the Data Team Analytics Project.
2. Choose a Template from the table below that best matches the work you are requesting.
3. Complete the templated Description as completely as possible.
4. Leave the Assignees blank. The Data Team will process your request as a part of our Daily

Triage.

<!--need to review the activities that are part of triage process -->

New Issue

| Request Type | Issue Template To Choose |

| ----- | ----- |

| General Data Team Support | [Request] Create Standard Data Team Issue (note: this is the default, most commonly used template for P3 / tactical requests and Data Team support. If you are not sure where to start, start here) |

| OKR-Level Project Request | [Request] Create Opportunity Canvas (note: this template is required for large-scale, P2 work like OKRs to be reviewed and prioritized in the Data Leadership Forum)|

| Add Data Source | [Request] New Data Source |

| Data Quality Issue | [Report] Data Quality Problem |

Data Leadership Forum

The Data Leadership Forum includes representation from partner teams across GitLab (Marketing, Sales, Customer Success, Finance, IT, Support, Product, Engineering, People, Security, Legal) and is used to oversee and drive the strategic direction of GitLab data management and analytics initiatives, including project prioritization. The forum ensures that data is leveraged effectively to support business goals, improve decision-making processes, and drive innovation. It acts as a governing body to establish policies, standards, and best practices for data governance, data quality, data privacy, and data security.

In order for OKRs / projects to be prioritized through the Data Leadership Forum, an opportunity canvas is required. An opportunity canvas is a specific issue template that contains detailed information about the work that is being requested, the expected business impact from that work, a rough estimate of the level of effort to accomplish the work, and known risks/dependencies. The opportunity canvas also includes a business value score based on our Value Calculator, which is one factor in prioritizing and ranking our backlog of work.

In the instance that a new project is raised mid-quarter and is proposed to be prioritized sooner than the next quarterly planning cycle, the forum will review the Opportunity Canvas for the new project, determine if the business value and impact warrants re-prioritization, and will come to a decision on the necessary trade-offs (i.e. to prioritize that new work, other planned work must be deprioritized).

Request to Expedite Responses

Requests to expedite responses, triage issues, or MR reviews are rare. Given the Data Team's shared-service model, expediting one task necessarily de-prioritizes other work. To request an expedited response:

1. Confirm there is a valid reason for moving your request ahead of others.

1. Post a note to data, along with a link to your Issue and a reason why you need an expedited response. Please do not DM an individual on the Data Team directly.

1. A member of the Data Team will respond within 1 business day.

Deciding What And How To Build

Not all data solutions require the same level of quality, scalability, and performance so we have defined a Data Development framework to help match required outcomes with level of investment. The Data Team works with all teams to build solutions appropriate to the need, but focuses on Trusted Data using Trusted Data Development.

Design Spike

Experimentation is a great approach to performing Explorational Data Development. Oftentimes, there can be multiple solutions to solve a business problem and we need a process to efficiently evaluate the different approaches and ultimately select a solution to promote to a Trusted Data solution. We use Design Spikes to facilitate experimenting. Design Spikes are particularly useful when the proposed solutions result in breaking changes or significant changes to the overall design, structure, and computing of the data tech stack.

The below steps should be followed when performing a Design Spike:

1. Calculate Value: Establish the Value the data solution can provide GitLab. Value can be measured in a variety of ways, ranging from efficiency, to increased Sales, to reduced compute.
1. Define Requirements: Create a Requirements document for the Design Spike to define the business and technical requirements the data solution must meet to be successful. Indicate whether each requirement is a Must Have or Nice to Have.
1. Experimentation Design: This is the most complex part of the Design Spike. We need to decide how to test the data solutions versus defined requirements. Oftentimes, successfully testing data solutions requires simulating production workloads. We need to define the minimal viable sample size of data to include in the design to achieve experimentation results that would be representative of the whole data set in scope for the solution.
1. Perform the experiment: This involves putting "fingers on the keyboard" and standing up a data solution in the Explorational data development environment. It also involves collecting the results of the experiment pursuant to what has been defined in the Define Requirements phase.
1. Assessment: After the experimentation is complete, we should perform an analysis of how well each data solution performed against the requirements from the Define Requirements phase and see how well each data solution performed against the Must Have or Nice to Have requirements. We should put this analysis into a document along with a recommended solution based on the results of the Design Spike. We can then submit this to the respective DRIs and Stakeholders on the project for review, feedback, and final decisions on how we will proceed with the use case.

Documentation

The Data Team, like the rest of GitLab, works hard to document as much as possible. We believe this framework for types of documentation from Divio is quite valuable. For the most part, what's captured in the handbook are tutorials, how-to guides, and explanations, while reference documentation lives within in the primary analytics project. We have aspirations to tag our documentation with the appropriate function as well as clearly articulate the assumed audiences for each piece of documentation.

Documentation guideline

There are several types of documentation we use to capture the topics. Noted the criteria when to use which type of the documentation

- (Public) Handbook - Items related to the operational model we used in the company and in the team, all explanations about tools, technologies and processes and why we are doing. Data that is publicly shareable, and does not expose GitLab or its customers to any harm or material impact (Green data, as per Data classification).
- Internal handbook - Items which explain the same category as the public handbook, with the difference that the internal handbook contains internal information
- Readme.md file - Specific information related to the code where the README.md file resides, which explains how to use that code. If more explanation is needed, a good practice is to either use and/or link to a Handbook article.
- Runbooks - Context which explains how to solve the issue in production or how to sort out other problems. The vital thing is to understand that runbook is a guideline of problem-solving approach
- WIKI - Items that require regular updates, similar to epics but they span for longer periods of time, e.g. overall Data Team engagement in the quote-to-cash projects.

Matrix with the explanation when to use which documentation type:

Example	Appropriate documentation type
GitLab Duo explanation	Handbook
Python/dbt/Snowflake guideline	Handbook
Description of the package inventory	Handbook
Explanation about the new pipeline/project	Internal Handbook
Data classification description	Internal Handbook
dbt Data lineage diagram	Internal Handbook
Technical explanation of how to run the project	README.md
Basic context about the project from the technical perspective	README.md
Solution of how to fix the Triage issue	Runbooks
Exploration article (ie. Design spike) of how to pseudonymize the data	Runbooks
Review current progress or effort on QtC projects	WIKI

Data Team Value Calculator

<!-- read through this and make adjustments -->

The Value Calculator provides a uniform and transparent mechanism for ranking and enables all work to be evaluated on equal terms. The value calculator approach is similar to the RICE Scoring Model for Product Managers and the Demand Metric Prioritization Model for Marketing.

The calculator below is based on the following Value Calculator spreadsheet. Please select the values below to define the value of new work.

Quarterly and Iteration Planning

Our planning process is called the Planning Drumbeat and it encompasses Quarterly Planning and Iteration Planning. The Planning Drumbeat is one of the most important activities the Data Team performs because it helps us align our work with the broader company, while remaining agile enough to manage shifting business priorities.

Quarterly KR Status Reporting

Beginning in FY25-Q1, the Data Team is using the GitLab Objectives and Key Results project to manage quarterly commitments.

Process:

1. Create KRs in the project for each of the committed Key Results.
2. In the KR description, add a link to the corresponding Epic from the GitLab Data Team project where the development work is being tracked.
3. Throughout the quarter, the DRI for the workstream should make the following updates (at a minimum, these updates should be added at the end of each month in the quarter for the KRs that have the Division:: scoped label applied to them; some teams may choose to make updates more frequently):
 - Add a comment outlining what work has been completed, and what work is remaining to complete the KR.
 - Update the % complete field on the KR.
 - Update the Health Status field to indicate whether the KR is On Track, Needs Attention, or At Risk.

DataPulse

DataPulse is a monthly rhythm for team members to provide status updates on Key Results and Business Operations initiatives. The goal is for everyone to stay informed about project statuses, identify new opportunities, and maintain alignment across our various workstreams.

Our process will have three key components:

1. Async written updates about the status of the Key Result and Business Operations

Monthly updates, captured in the Key Result or Business Operations issues and epics (this part we are already doing and will not change):

- Status of project work
- Challenges and proposed solutions

2. Asynchronous Video Updates (Due 1 week before Monthly Q&A)

For Key Result and Business Operations DRIs

Each DRI (and team) will record a brief, maximum 5-minute, video update covering:

- Intro/context about the business problem that we addressing in the Key Result or Business Operations
- Status update
- Challenges and proposed solutions
- Business stakeholder feedback and engagement
- New opportunities identified

Within your KR and Business Operations teams, please develop a plan for these monthly updates. As the DRI, you're accountable for aligning on this approach with your project team.

Recording Guidelines:

- These videos don't need to be perfect - aim for a conversational tone.
- Consider using time in an existing meeting with the KR team to create this recording. No slides / materials are required.
- Include mentions of team members who are part of your work.
- Upload your video to our Data Team YouTube playlist one week before our scheduled Q&A session. Include the month and year in the title of the recording and add a link to the video in the Key Result or Business Operations epic/issue.
- This is your chance to share your work with team members who you don't collaborate with on a regular basis - take advantage of it!

3. Monthly Q&A Session (6am PST / 2pm PST options)

We'll meet for a focused Q&A session to discuss the updates shared in the videos. This allows us to:

- Dive deeper into specific areas of interest
- Address questions that arise from the video updates
- Collaborate on resolving challenges
- Align on next steps

Introducing a new data source

<!-- Amie to check with Dennis on this -->

Introducing a new data source requires a heavy lift of understanding that new data source, mapping field names to logic, documenting those, and understanding what issues are being delivered.

Usually introducing a new data source is coupled with replicating an existing dashboard from the other data source.

This helps verify that numbers are accurate and the original data source and the data team's analysis are using the same definitions.

Incidents

Incidents are times when a problem is discovered and some immediate action is required to fix the issue. When this happens, we make an Incident Issue in the Data Team Project. The process for working through incidents is as follows:

- Open an Incident issue using the "Incident Report" template
- Detail the relevant information with appropriate timestamps
- Tag and assign people on the Data Team and any other teams that need to be informed
- Review the Security Team's documentation on Incident Response and take any necessary action

Data Team Incidents can be reviewed in Incident Overview page within the main project.

Workflow Summary

Stage (Label)	Responsible	An Item Is Added to This Stage When
Criteria to Progress to Next Stage		

Workflow	Description	Notes
workflow::1 - triage & validation	Data Triager	A new request has been created A clear problem statement & business value statement and expected outcome are included in the issue, and appropriate labels (Work Category, Champion, and Team) have been applied. The issue has a numerical weight applied by the Data Triager. If the work does not warrant development, a description for why the work won't be done will be added, and the issue will be closed.
workflow::2 - waiting for prioritization	Data	The issue is scoped, sized and warrants development. Work is prioritized and implementation timelines are agreed upon.
workflow::3 - refinement	Data, Business DRI	The issue is actively being refined The technical solution have enough detail and clarity that (another) developer (other than the one doing the refinement) would be able to pick it up.
workflow::4 - ready to develop	Data	The issue is fully scoped & refined Work is picked up for development
workflow::5 - development	Data	Development work has started Item is actively being worked on.
workflow::6 - review	Data, Business DRI	Development work is ready for, or currently, being reviewed All work is completed.
workflow::X - blocked	Data, Business DRI	Issue needs intervention that assignee can't perform Work is no longer blocked

Generally issues should move through this process linearly. Some templated issues will skip from triage to scheduling or scheduled.

Issue Pointing

Issue pointing captures the complexity of an issue, not the time it takes to complete an issue. That is why pointing is independent of who the issue assignee is.

- Refer to the table below for point values and what they represent.
- We size and point issues as a group.
- Effective pointing requires more fleshed out issues, but that requirement shouldn't keep people from creating issues.
- When pointing work that happens outside of the Data Team projects, add points to the issue in the relevant Data Team project and ensure issues are cross-linked.

Weight	Description
Null	Meta and Discussions that don't result in an MR
0	Should not be used.
1	The simplest possible change including documentation changes. We are confident there will be no side effects.
2	A simple change (minimal code changes), where we understand all of the requirements.
3	A typical change, with understood requirements but some complicating factors

- |
- | 5 | A more complex change. Requirements are probably understood or there might be dependencies outside the data-team. |
- | 8 | A complex change, that will involve much of the codebase or will require lots of input from others to determine the requirements. |
- | 13 | It's unlikely we would commit to this in an iteration, and the preference would be to further clarify requirements and/or break into smaller Issues. |

Issue Labeling

Think of each of these groups of labels as ways of bucketing the work done.

All issues should get the following classes of labels assigned to them:

- Team: The Data Team responsible for performing the work. This could be one of the following: Data Platform, Analytics Engineering, Data Science, BI, or Data Governance.
- Champion: The team requesting the work. This may be a functional partner team or the Data Team itself.
- Workflow: The current status of the work. All issues should start with the workflow::1 - triage label. The team performing the work is responsible for updating the workflow status as the work progresses.
- Priority: The priority level of the work, categorized as follows:
 - P1: Operational (highest urgency)
 - P2: OKR-related work (objective-driven)
 - P3: Other (lower priority or non-urgent tasks)

Effective in January 2025, we use a Bot on the Data Team project to check that the Team, Champion, Workflow, and Priority labels have been applied to issues after 14 days of being opened. The bot sends a reminder in the issue to add the missing labels. The first triage response for adding labels is the team member that opens the issue. The 2nd triage response for adding labels is the Data Analyst, Data Scientist, Analytics Engineer, and Data Engineer that are on triage. Issues that do not have Team, Champion, Workflow, and Priority labels applied after 30 days are automatically closed. If an issue is closed due to not having the required labels, team members have the option to reopen the closed issue and apply the labels to meet the issue refinement requirements.

Optional labels that are useful to communicate state or other priority:

- What:
 - Data: Data being touched (Salesforce, Zuora, Zendesk, GitLab.com, etc.)
 - Tool: (Tableau, dbt, Stitch, Airflow, etc.)
 - Pod: Data team pod that is scheduling the work
- Business Logic Change: This label is applied for any business logic changes such as adding new dimensions, facts, marts, changing joins, adding new calculated fields.
- Opportunity Canvas: This label is auto-applied on the Opportunity Canvas template, but can also be applied to work that has converted into a large-scale project. This label will be used to identify topics for discussion and prioritization at the monthly Data Leadership forum.

Epic Labeling

Similar to issue labeling, epic labeling helps the Data team categorize, quantify, and prioritize the projects in our backlog.

At a minimum, all epics should have a Team: label applied. This tags the epic for the Data Team that is primarily responsible for performing the work, and enables managers to review the backlog of projects for their respective teams. This is particularly helpful during quarterly planning.

Because the epic list cannot easily be filtered to parent epics only (and epics may be nested under other epics as sub-epics), we use an additional label to distinguish between epics that are being considered for OKR-level commitments (these should have the Opportunity Canvas label applied) vs. those that are used to group related issues under a general theme (these should not have the Opportunity Canvas label applied).

Merge Request Workflow

Ideally, your workflow should be as follows:

1. Confirm you have access to the analytics project. If not, request Developer access so you can create branches, merge requests, and issues.
1. Create an issue, open an existing issue, or assign yourself to an existing issue. The issue is assigned to the person(s) who will be doing the work.
1. Add appropriate labels to the issue (see above)
1. Open an MR from the issue using the "Create merge request" button. This automatically creates a unique branch based on the issue name. This marks the issue for closure once the MR is merged.
1. Push your work to the branch
1. Update the MR with an appropriate template. Our current templates are:
 - dbt Changes - used for any change involving dbt. Analysts will most often use this one
 - addmanifesttables - for adding tables to pgp extract
 - periscope - for getting a Periscope dashboard reviewed
 - pythonchanges - for general changes to Python code
 - All Other Changes - for work that doesn't generally fall into these categories
1. Run any relevant jobs to the work being proposed
 - e.g. if you're working on dbt changes, run the job most appropriate for your changes. See the CI jobs page for an explanation of what each job does.
1. Document in the MR description what the purpose of the MR is, any additional changes that need to happen for the MR to be valid, and if it's a complicated MR, how you verified that the change works. See this MR for an example of good documentation. The goal is to make it easier for reviewers to understand what the MR is doing so it's as easy as possible to review.
1. Request a review by assigning the MR to a peer using the Merge Request Reviewer feature.
 - Requesting a review in this manner indicates to the person that you would like their code review and approval if everything is good. This does not mean they will merge the MR if they approve it.
 - The peer reviewer should use the native approve button in the MR after they have completed their review and approve of the changes in the MR.

- After approval, the reviewer can unassign themselves from the Reviewer list. The reviewer is not responsible for the final tasks. The author is responsible for finalizing the checklist, closing threads, removing Draft, and getting it in a merge-ready state.

1. If the MR is approved, remove the Draft: label, mark the branch for deletion, mark squash commits, and assign to the project's maintainer. Ensure that the attached issue is appropriately labeled and pointed.

- Generally, assigning the MR to a maintainer indicates you would like for them to merge it if there are no issues. If the maintainer needs to approve the merge request before merge as part of a CODEOWNER group, then they will do a review before merging. Otherwise, they will simply merge. If you would like the maintainer's review regardless, simply leave a comment to that effect.

- Note that assigning someone an MR means action is required from them.

- If Draft: is still in the title of the MR, then the Maintainer will assign the MR back to the author to confirm that the MR is ready for merge.

Other tips:

1. The Merge Request Workflow provides clear expectations; however, there is some wiggle room and freedom around certain steps as follows.

- For simple changes, it is the MR author who should be responsible for closing the threads. If there is a complex change and the concern has been addressed, either the author or reviewer could resolve the threads if the reviewer approves.

1. Reviewers should have 48 hours to complete a review, so plan ahead with the end of the iteration.

1. When possible, questions/problems should be discussed with your reviewer before submitting the MR for review. Particularly for large changes, review time is the least efficient time to have to make meaningful changes to code, because you've already done most of the work!

1. Consider bringing the latest commits from the primary branch so the MR is caught up. You can do this quickly by typing `/rebase` into a comment and GitLab will make this happen automatically, barring any merge conflicts.

KPI Development Workflow

The Data Team will work to add KPIs and Performance Indicators to our enterprise database models and BI reports once the following steps have been completed.

- The DRI should ensure the KPI definition, business logic, and calculation steps are documented in the relevant section of the handbook and added to the GitLab KPIs with all of its parts

- The handbook definition should be reviewed by the necessary Consulted & Informed cross-functional partners. In some cases, the definition may also require approval from those cross-functional partners

- Once the KPI is ready to be added into our enterprise reporting, the DRI should create an issue using the standard Data Team Issue template on the GitLab Data Team Issue Tracker.

- The Data team will verify the data sources and help to find a way to automate (if necessary).

Once the KPI has been added to our enterprise BI platform, the Data Team will present it to the DRI for testing and final approval.

SLO for Issues and Merge Requests

- First-Response SLO for a new Issue: 2 business days from the time of issue creation
- First-Response SLO for a new MR: 2 business days from the time of submission to the codeowner

When opening an issue or submitting a MR for review, it is good to add a comment bringing the issue or MR to the attention of a DRI or codeowner. Please allow the SLO time period to expire before requesting additional updates or first responses on an Issue or MR. Issues and MRs within the SLO time period are not considered blocked. If the issue or MR is urgent or a break-fix scenario, it is good to follow-up with the team member in the Issue or MR within the SLO period as determined by the required urgency and importance.

Removal and deletion process

<!-- @dennis is there an iteration on this one, get @chris and @peter to contribute here -->

After some time, environments will have software/code/components that are not needed any more. It feels risky to delete software and code, even when its not being used, seems not being used or asked not to being used (i.e. users access).

There are multiple reasons to perform deletions:

1. We see software (or parts of software) that is not used anymore
2. There is a request to remove user access
3. There is a request to delete a user account
4. There is a request to indefinitely stop a data pipeline

To address observations and requests, and ensure that deletion will take place in a controlled manner, open an issue with the Cleanup Old Tech template.

State down in the issue the scope of deletion

Write down what will be deleted and where possible link to existings issues.

Calculate a Risk score

The Risk score is build upon 2 variables.

1. Probability that it will break something
2. Impact if the deletion is executed by mistake or is execute wrongly

Each variables will be scored 1 to 3.

Probability	Score
Low	1
Medium	2
High	3

Impact	Score

Negligible	1
Lenient	2
Severe	3

Probability Impact = Risk Score

```
<style type="text/css">
.tg {border-collapse:collapse;border-spacing:0;}
.tg td{border-color:black;border-style:solid;border-width:1px;font-family:Arial, sans-
serif;font-size:14px;
    overflow:hidden;padding:10px 5px;word-break:normal;}
.tg th{border-color:black;border-style:solid;border-width:1px;font-family:Arial, sans-
serif;font-size:14px;
    font-weight:normal;overflow:hidden;padding:10px 5px;word-break:normal;}
.tg .tg-1i2r{background-color:f8ff00;border-color:inherit;text-align:left;vertical-align:top}
.tg .tg-0pky{border-color:inherit;text-align:left;vertical-align:top}
.tg .tg-y698{background-color:efefef;border-color:inherit;text-align:left;vertical-align:top}
.tg .tg-706m{background-color:009901;border-color:inherit;text-align:left;vertical-align:top}
.tg .tg-tw5s{background-color:fe0000;border-color:inherit;text-align:left;vertical-align:top}
</style>
<table class="tg">
<thead>
<tr>
<th class="tg-0pky"></th>
<th class="tg-0pky">Risk</th>
<th class="tg-0pky">Negligible</th>
<th class="tg-0pky">Lenient</th>
<th class="tg-0pky">Severe</th>
</tr>
</thead>
<tbody>
<tr>
<td class="tg-0pky">Probability</td>
<td class="tg-y698"></td>
<td class="tg-y698"></td>
<td class="tg-y698"></td>
<td class="tg-y698"></td>
</tr>
<tr>
<td class="tg-0pky">Low</td>
<td class="tg-y698"></td>
<td class="tg-706m">1</td>
<td class="tg-706m">2</td>
<td class="tg-1i2r">3</td>
</tr>
<tr>
<td class="tg-0pky">Medium</td>
<td class="tg-y698"></td>
<td class="tg-706m">2</td>
<td class="tg-1i2r">4</td>
```

6
High
3
6
9

Risk Score	Outcome
1 - 2	Create a MR and have it reviewed by 2 code owners
3 - 4	Create a MR, tag @gitlab-data/engineers with a deadline to object and have it reviewed by 2 code owners
6 - 9	Create a MR, to be discussed in the DE-Team meeting and have it reviewed by 2 code owners

Annual Data & Analytics Maturity Assessment

Each year beginning in FY24, the Data Team will facilitate a survey to solicit anonymous feedback about the Data program from GitLab team members. This survey will help drive our focus areas and track our maturity over time.

YouTube

We encourage everyone to record videos and post to GitLab Unfiltered. The handbook page on YouTube does an excellent job of telling why we should be doing this. If you're uploading a video for the data team, be sure to do the following extra steps:

- Add data as a video tag
- Add it to the Data Team playlist
- Share the video in data channel on slack

Data Hiring and Interview Process

1. Review the People Interviewing Guide to ensure you are up-to-date on the current process and policies.

1. Contact your People BP (Business Partner) and alert them of the opening. The People BP is responsible for managing the job opening, requisition, and interview processes.

1. Ask your People BP to provide you a Requisition for tracking purposes. If you have multiple openings to fill, communicate with your People BP using this Req because juggling multiple openings gets confusing fast.

1. Develop an Interview Plan, which will cover Responsibilities, Tips, Reminders, and custom Interview questions for each Interview Job Role type. For examples, see the Data Scientist Interview Plan and Analytics Engineer Interview Plan.

1. Each Data Job Role and Job Grade has a customized Homework Assessment. Review and update the Homework Assessment as needed. If a Homework Assessment is not available for the Job Role,

create one and save it in the Homework Assessment Google Drive

1. Send a Homework Assessment to the People BP for inclusion in the Hiring process. The Hiring Process is included in each Data Job Family to help set candidate and interviewer expectations. An example is the Data Engineer Hiring Process.

1. Create a new slack channel, e.g. bt-data-data-science-interview to help coordinate with your interviewers. Share the Interview Plan with your interviewers through Slack.

SECTION: enterprise-data/how-we-work/calendar.md

Meetings

The Data Team's Google Calendar is the SSOT for team meetings.

It also includes relevant events in the data space.

Anyone can add events to it.

Many of the events on this calendar, including Monthly Key Reviews, do not require attendance and are FYI events.

When creating an event for the entire Data Team, it might be helpful to check their working hours in Google Calendar and discuss out of working hour meetings ahead of scheduling. Please consider alternating who is meeting after working hours when such meetings are necessary.

No-Meetings Fridays

While we can't promise we'll never have meetings on Fridays, the team has adopted No Meetings Fridays.

Bias For The Maker's Schedule

Most Data Team members identify more strongly with the Maker's Schedule over the Manager's Schedule and we support this by:

- limiting individual contributor meeting exposure to no more than 20% of their time
- grouping our meetings into contiguous chunks instead of sprinkling them throughout the day or week
- holding most of our required recurring meetings on Tuesdays and Thursdays

Meeting Protocol

The team follows GitLab meeting practices and standards.

- Every meeting has an agenda.
- Meeting topics are listed in priority order.
- Notes are added to the agenda inline with topics and questions.
- All agendas are stored in the Data Team Meetings shared drive.

Recurring Meetings

Daily Standup

Members of the data team use Geekbot for daily or weekly standups.

These standup notes are posted in data-daily and/or data-weekly.

When Geekbot asks, "What are you planning on working on today? Any blockers?" try answering with specific details, so that teammates can proactively unblock you.

Instead of "working on Salesforce stuff", consider "Adding Opportunity Owners for the sfdcopportunityxf model."

There is no pressure to respond to Geekbot as soon as it messages you.

Give responses to Geekbot that truly communicate to your team what you're working on that day, so that your team can help you understand if some priority has shifted or there is additional context you may need.

SECTION: enterprise-data/how-we-work/dbt-change-workflow.md

dbt Change Workflow Summary

To facilitate high quality and confidence in changes to dbt models and documentation, the following workflow has been established. Following this workflow provides developers and reviewers with the information needed to successfully complete their tasks. It leans on the DevOps framework as well as Test-Driven Development principles to give developers and reviewers information and structure to the dbt change process.

mermaid

flowchart LR

subgraph dbt Changes

direction LR

subgraph Plan

direction TB

a("Define Scope") --> b("Define Tests") --> c("Define Environment")

end

subgraph Create

direction TB

d("Build

Local Environment") --> e("Change Code") <--> f("Test Locally")

end

subgraph Verify

direction TB

g("Build

Remote Environment") --> h("Test Remotely") --> i("Review")

end

end

Plan --> Create

Create <--> Verify

1. Plan - This stage defines how development and reviews will take place. The output of this stage is a Merge Request filled out with a plan to guide subsequent stages.

1. Define Scope - This step describes the changes that will be made. Details can be deferred to an issue or epic.

1. Define Tests - This step defines how the change will be checked.

1. Define Environment - This step defines the models that will be used to develop and check the changes.

1. Create - This stage is where code changes take place. The code changes are then validated using tests defined during the Plan stage. The output of this stage is a set of developer-verified code changes, as defined by the plan.

1. Build Local Environment - Using the environment defined in the plan, tables are loaded in the developer's local database.

1. Change Code - Using the planned scope, changes are made to the code.

1. Test Locally - Using the tests defined during planning, the changes are verified and validated.

1. Verify - This stage is where the completed changes are validated in a deployment-like environment and reviewed by other qualified developers. The output of this stage is code changes and test results that have been reviewed and approved.

1. Build Remote Environment - Using the environment defined in the plan, tables are loaded in a remote database.

1. Test Remotely - Using the tests defined during planning, the changes are verified and validated.

1. Review - The results of the remote testing are compared against the scope and tests defined in the plan to assure quality of the suggested code changes.

Determining Degree of Testing and Review

As this workflow is given as a general process, depending on the complexity of the changes being made and the location of where the changes are may result in a higher or lower degree of testing and review at any given step. The degree of testing and review should be determined during planning by the developer so that the correct amount of effort is placed into each step.

The first step in determining the degree of testing and review is identifying the location of the changes which will provide a minimum degree of testing and review. From there the type, volume, and impact of the change will need to be considered to determine the final degree of testing and review.

Location Testing and Review Categories

- High
 - The Enterprise Dimensional Model (PROD.COMMON)
 - Key Performance Indicator Reporting (e.g. martarr)
- Medium
 - Legacy models
 - Source models
 - Preparation models
- Low
 - Workspace Models (e.g. PROD.WORKSPACESALES.)

Changes Testing and Review Categories

- High
 - Integrating new source data into existing models
 - Changing a model that is referenced by many other models
- Medium
 - Adding models for a new data source
- Low
 - Changing a column name
 - Updating documentation

dbt Change Workflow Details

Define Scope

The scope serves as context for the changes to be made, and it aids the developer and the reviewers in understanding the intent of the change. Changes to table and view models, and their documentation, are considered typical. Any other changes may require special testing or processing, and should be identified during the planning process. The list below identifies known atypical changes and the changes or exceptions related to those types of changes:

- changes to macros
 - Requires that models that use the macro be tested even if the models are not directly changed.
- changes to incremental models
 - Requires that the model be configured for schema changes or full refreshed after the changes are merged. Note that performing a manual full refresh may collide with orchestrated executions of dbt and as such a manual refresh should only be performed after the orchestrated execution of the models.
- changes to seeds
 - Requires that the loading of the seed file be explicitly, using a full refresh, performed as part of the testing.
- changes to snapshots
 - Requires that changes are all tested in the remote environment.
- changes to project configuration
 - Requires that a model be tested to ensure that the project compiles.
- changes to documentation only
 - Requires that a model be tested to ensure that the project compiles.
- changes to workspace models
 - Only requires base level component testing

Define Tests

Before changes are made to the code, the tests needed to check that the changes have the intended impact are to be developed first. This follows software development principles of Test Driven Development and will aid the developer in outlining what changes need to be made and how those changes will be evaluated as successful. To aid in communication, tests can be categorized into several types:

- Component Tests
- Integration Tests
- System Tests
- Acceptance Tests

Component Tests

Component tests check that each model changed correctly builds independently of other models. At a minimum, any model with a change needs to be executed after the changes are made to ensure that the code compiles and executes without error, this is the most basic component test. Other expected component tests when building tables in the database include testing for uniqueness within the table and nulls in specified columns. Other component tests may include: testing the addition or removal of a column, testing the change to or addition of filters, or testing the addition or removal of business logic. A description of the types of component tests that will be performed on the changed models is a required part of the Plan stage, however a detailed list of every component test that will be used is not needed.

Integration Tests

Integration tests check that a changed model does not negatively impact the operation of any other model. Any model that depends on a changed model should be built and component tests executed as part of the integration testing of a changed model. However, when the expected sequential build time of all of the dependent models is excessively long, more than an hour of run time, then a subset of the dependent models can be selected. When selecting a subset of models, the longest or most critical path should be selected while minimizing the sequential build time. Additionally, if the changes are net new, such as adding a new column, then integration tests can be omitted with the following exceptions:

- A downstream model is configured to be incremental and not configured to append new column or sync all columns when the schema changes

All incremental downstream models of a change model that would need to be refreshed to prevent an error at run time can be found using the following dbt command:

```
console
dbt list -s <changed model>+,config.materialized:incremental --output name --resource-type
model --exclude config.onschemachange:syncallcolumns config.onschemachange:appendnewcolumns
```

All downstream models of a changed model can be found using the following dbt command:

```
console
```

```
dbt list -s <changed model>+ --output name --resource-type model
```

System Tests

System tests check the output of changed models against the output of existing models. Basic system tests may include comparing row counts of the changed and current version of the model.

A basic system test to compare the output of the development table with the production table might look like this:

```
sql
```

```
WITH compare AS (  
  SELECT  
  FROM database.schema.table  
  
  MINUS  
  
  SELECT  
  FROM <branchname>database.schema.table  
)
```

```
SELECT COUNT() from compare
```

Acceptance Tests

Acceptance tests can be defined by Stakeholders or Subject Matter Experts and will be aimed at validating that the changes are meeting the needs of the users of the resultant models.

Define Environment

To correctly construct and test the models that will be changed, or models affected by the changes, many models may be necessary. These models can be classified as environment models or build models, and a full list of all models needed is required for the Plan stage.

Environment Models

Environment models are those needed to build or test the models changed or affected by the changes. Once these models have been created within the appropriate environment, via a clone or construction, they do not need to be updated during the development process. A dbt list command that will include all of the environment models is a required part of the Plan stage.

Example commands:

- 1+<changed model> Includes changed model and the models that come immediately before the changed model.
- Recommended for simple or direct changes with low to no integration testing.

- +<change model> Includes the changed model and the models that come before the changed model.
 - Recommended when making significant changes to a full chain of models with low amounts of integration testing.
- @<changed model> Includes the changed model, all of the models that come before the changed model, all of the models that come after the changed model, and all of the models that come before the models that come after the changed model.
 - Recommended for used with extensive integration testing as it will ensure that the relevant models are available to reference during testing.

mermaid

flowchart LR

```

subgraph dbt Model Lineage
  subgraph "@Changed Model"
    direction LR
    subgraph "+Changed Model"
      subgraph "1+Changed Model"
        b("Model B") --> t(["Changed Model"])
        z("Model Z") --> t(["Changed Model"])
      end
      a("Model A") --> b("Model B")
      y("Model Y") --> z("Model Z")
      x("Model X") --> y("Model Y")
    end
    t(["Changed Model"]) --> l("Model L") --> m("Model M")
    j("Model J") --> k("Model K") --> l("Model L")
  end
end
end

```

Build Models

Models that will be reconstructed as part of building or testing changes are classified as build models. These models contain the code changes or are models affected by the code changes and will need to be re-created multiple times during the development process.

Prepare Local Environment

Using the environment defined during the planning, construct the local development environment using the following steps:

1. Clean local database tables by dropping all schemas found with in the <username>PREP and <username>PROD. This will ensure that the data used for testing is only the data added recently. This can be done through the Snowsight UI or using a SQL command and query details can be found in the Snowflake Dev Clean Up runbook.
1. Using the environment models command, clone tables necessary for development and testing. This ensures that the data is up to date, and cloning the tables is more efficient than rebuilding the tables from the source data.
 - The clone job will not clone under the following conditions. Any models meeting those

conditions must be included in the subsequent build steps:

- The table is new
- The table has a new name
- The table is an external table
- The table is a partition other than the current partition

1. If the build models require more 10 minutes to build on an XL warehouse then the environment models directly used for the build models should be sampled. Sampling the data will improved the build time and increase the iterations that can be performed during development.

Change Code

The Change Code step is where any draft changes are translated into language and format used in the code repository. The process starts with documentation, keeping with the spirit of Handbook First, and iterates through the Test Change Locally step until the changes pass the tests added in the documentation and the target changes are effected. As part of this step the SQL style guide and linter must be used so that there is a consistent form and feel to all of the SQL used in the dbt code base.

Test Changes Locally

Test all components, integrations, and system tests locally to ensure that the changes have been implemented as planned. Any test failure requires an iteration cycle with the change code step until all tests pass. If the data was sampled, the sampling does not need to be removed for this stage of testing and is left to the discretion of the developer to determine if testing on the full data is needed. Using the build command, instead of separate run and test commands, during development will efficiently run inbuilt component and integration tests and is recommend best practice.

Prepare Remote Environment

To ensure an isolated environment, documentation of results, and to improve collaboration during reviews, all tests must be run in a remote environment. Using the environment defined during planning, construct the remote environment using the provided CI Jobs.

1. Using the environment models command, clone tables necessary for development and testing.

- If the environment models command only contains models with direct changes and their ancestors, then the `runchangedmodelssql` CI job can be used.

- The clone job will not clone under the following conditions. Any models meeting those conditions must be included in the subsequent build steps:

- The table is new
- The table has a new name
- The table is an external table
- The table is a partition other than the current partition

Test Changes Remotely

Test all components, integrations, and system tests remotely to ensure that the changes have been implemented as planned. Any test failure requires an iteration cycle with the Change Code step until all tests pass.

1. Using the build models command from the defined environment build the changed models. This will build the changed models and perform any tests that have been included with the changes.

- This can be performed with one of the specifymodel CI Jobs. If the build command only contains models with direct changes then one of the runchangedmodelssql CI jobs can be used.

- If the changed models are incremental models, the models run more than one hour on a full refresh, and a full refresh does not need to be explicitly tested then the full refresh overridden should be used by adding the REFRESH = variable.

1. Use the rungrants CI job, to grant permissions to yourself and run any remaining components, integrations, and system tests on the build models, adding the results of the tests to the MR.

- If the table is new then the grantclones job will not be able to grant access the table.

In this case documentation of local tests are required for verification and validation.

Reviews

All need to be reviewed for accuracy of the code changes and there effects as well as compliance to our standards and practices. While not every review will be needed for every change and individuals can perform multiple reviews simultaneously, a Code Owner Review and a Maintainer Review will always be required for each change.

- Accuracy and Effects
 - Stakeholder Review
 - Subject Matter Expert Review
 - Code Owner Review
- Compliance
 - Maintainer Review

Example

An example of how this workflow can be used can be seen in MR 8476 where columns were renamed for a single table.

Plan

Scope

Update the output column names to match the end user needs.

Tests

In addition to testing for uniqueness the column names will be checked to ensure that they match the requested column name.

Environment

The planned changes need the following models to properly build and test the changes:

- pumpdisasterrelieffund
- employeeedirectory

Environment

```
console
dbt list 1+pumpdisasterrelieffund
```

Build

```
console
dbt list pumpdisasterrelieffund
```

Create

Build Local Environment

The following command was used to build the local environment

```
console
make DBTMODELS="1+pumpdisasterrelieffund" clone-dbt-select-local-user
```

Resulting in the "PEMPEYPREP"."SENSITIVE"."EMPLOYEEEDIRECTORY" and "PEMPEYPROD"."PUMPSSENSITIVE"."PUMPDISASTERRELIEFFUND" being created

Change Code

While making the changes the following command was used to build and test the changes

```
console
dbt build -s pumpdisasterrelieffund
```

Resulting in the PEMPEYPROD.pumpssensitive.pumpdisasterrelieffund being build and the dbtutilsuniquecombinationofcolumnspumpdisasterrelieffundEmployeeenumberActionTrue test begin executed.

Test Locally

In addition to the built in tests the following queries were used to test the changes.

<details>

<summary>SQL</summary>

```
sql
SET queryid = (
  SELECT QUERYID
  FROM snowflake.accountusage.queryhistory
  WHERE true
  and QUERYTEXT = 'SHOW COLUMNS IN pempeyprod.pumpssensitive.pumpdisasterrelieffund'
  AND starttime > currentdate())
```



```

ORDER BY STARTTIME DESC
LIMIT 1
);

SELECT $queryid
--

SELECT count() = 6
FROM TABLE (RESULTSCAN($queryid)) --
WHERE true
AND "columnname" IN ('Fund','Firstname','Lastname','Employeeenumber','Email','Action')

-- Integration Test

-- None

-- System Test

WITH compare AS (
  SELECT
    FROM prod.pumpssensitive.pumpdisasterrelieffund

  MINUS

  SELECT
    FROM pempeyprod.pumpssensitive.pumpdisasterrelieffund
)

SELECT count() from compare

```

</details>

Verify

Build Remote Environment

The clonemodeldbtselect job was used with the DBTMODELS = 1+pumpdisasterrelieffund variables to build the environment.

Test Remotely

1. First the specifymodel job was used with the DBTMODELS = pumpdisasterrelieffund variables to build and test the changes.
1. Second the grantclones job was used with the GRANTTOROLE = PEMPEY variable to grant query access to the changes.
1. Third the following queries were executed and the results added to the MR:

<details markdown="1">

<summary>SQL</summary>

sql

```
SET queryid = (  
  SELECT QUERYID  
  FROM snowflake.accountusage.queryhistory  
  WHERE true  
  and QUERYTEXT = 'SHOW COLUMNS IN 17086-ADJUST-COLUMN-NAMES-FOR-E4E-RELIEF-  
FILEPROD.pumpssensitive.pumpdisasterrelieffund'  
  AND starttime > currentdate()  
  ORDER BY STARTTIME DESC  
  LIMIT 1  
);
```

```
SELECT $queryid  
--
```

```
SELECT count() = 6  
FROM TABLE (RESULTSCAN($queryid)) --  
WHERE true  
AND "columnname" IN ('Fund','Firstname','Lastname','Employeeenumber','Email','Action')
```

```
-- Integration Test
```

```
-- None
```

```
-- System Test
```

```
WITH compare AS (  
  SELECT  
  FROM prod.pumpssensitive.pumpdisasterrelieffund
```

```
  MINUS
```

```
  SELECT  
  FROM 17086-ADJUST-COLUMN-NAMES-FOR-E4E-RELIEF-FILEPROD.pumpssensitive.pumpdisasterre  
)
```

```
SELECT count() from compare
```

</details>

Review

One of the code owners was assigned as a reviewer. The code owner was also a maintainer and performed both reviews finally merging the changes.

SECTION: enterprise-data/how-we-work/mr-review.md

Merge Request Roles and Responsibilities

Reviewer

All GitLab data team members can, and are encouraged to, perform code review on merge requests of colleagues and community contributors.

If you want to review merge requests, you can wait until someone assigns you one, but you are also more than welcome to browse the list of open merge requests and leave any feedback or questions you may have.

Note that while all team members can review all merge requests, the ability to accept merge requests is restricted to code owners.

The responsibility of a reviewer is

- to review the technical implementation,
- to ensure the code achieves the business goals,
- to check the data quality of the data models created.

Code owner

Code ownership is a feature of GitLab that links a project member to specific folders and files in a project. It is meant to answer the questions "who can I ask about this code?" and "who should review changes to this code?". The goal is to assign 2 or 3 GitLab Team Members (max 4) to a set of folders and files (if possible).

With not assigning a big group of people to basically all of our code, we hope to define better ownership and drive better code reviews. To keep the efficiency we potentially will assign more code owners to Snowflake Workspace schemas since the workspaces are not subject to higher quality code reviews.

A code owner:

- is the first contact person to answer questions regarding the code,
- carries the ownership of the quality of the code,
- carries the ownership of the results that the code produces,
- performs MR reviews and accepts merge requests.

How to become a code owner

1. Create an MR to the CODEOWNERS file with the ownership change you wish to make
 - Explain in the MR body why they are ready to take on that responsibility.
 - Use specific examples of recent MR reviews that they have performed. MRs that introduce very simple changes are good, but should not be the only source of reviews.

1. Work with the other code owners that already cover the area you wish to join.

Requirements that should be taken into consideration to become a code owner:

- You should have an advanced understanding of the particular subject area.
- The MRs you've written consistently make it through reviewer and maintainer review without significant required changes.
- The MRs you've reviewed consistently make it through maintainer review without significant additionally required changes.

Maintainer

A maintainer in any of the data team projects is not synonymous with any job title.

Here, the data team takes from the precedent set forward by the engineering division on the responsibilities of a maintainer.

Every data team project has at least one maintainer, but most have multiple, and some projects (like Analytics) have separate maintainers for dbt and orchestration.

The responsibility of a Maintainer is to ensure that

- Data team processes are followed,
- the MR is not conflicting with the broader Data Team architecture, procedures and processes,
- final review on the MR.

How to become a data team maintainer

We have guidelines for maintainership, but no concrete rules.

Maintainers should have an advanced understanding of the GitLab Data projects codebases.

Prior to applying for maintainership of a project, a person should gain a good feel for the codebase, expertise in one or more domains, and deep understanding of our coding standards. You're probably ready to become a maintainer when both of these statements feel true:

1. You are already a codeowner in multiple subject areas
1. The MRs you've reviewed consistently make it through maintainer review without significant additionally required changes
1. The MRs you've created consistently make it through reviewer and maintainer review without significant required changes

If those subjective requirements are satisfied, this is the process to add yourself as a maintainer:

1. Create an issue in the relevant project with the title "Add <user> as project maintainer"
1. Add documentation to the issue for the following:
 - Explain why you are ready to take on the maintainer responsibility
 - Explain the scope of your maintainership (entire project, dbt, python, etc.)
 - Recent MR's that you have created and reviewed that you believe show your readiness
1. Once the issue is created, tag all other maintainers
1. If denied, close the issue and work with your manager for a period of no less than 3 months before applying again
1. If approved, create a MR to add the maintainership to your team page entry

1. Assign the MR to your manager and mention the existing maintainers of the relevant project (Infrastructure, Analytics, etc) and area (dbt, Airflow, etc.).
1. If the existing maintainers do not have significant objections we've got ourselves a new maintainer!
1. An owner of the project will increase your privilege on the project

Merge Request Workflow

Every MR follows the applicable MR template. When a MR is ready for review, assign a code owner for review. When a code owner approves the MR, the MR can be merged by a maintainer. This could be the same person if the code owner is also a maintainer. If not, a maintainer is tagged in the MR, asking for final review and merge.

Merge Request approval requirements

Because of quality and security we require every MR to be approved before merged with multiple Team Members involved. By default the CODEOWNER file sets that an approval is needed and by who. But if a file or folder is not present in the CODEOWNER file a MR still can be merged without approval or without other Team Members involvement. To enforce the following 2 settings must be applied on projects within the GitLab-Data Group if it meets the criteria mentioned in this handbook section.

1. Approvals required = 1
setti